

AI (SW) Factories*

Forkable Sandboxes as a Runtime for
Automated Search and Development

Yossi Eliaz

Principal Engineer, Incredibuild / islo.dev

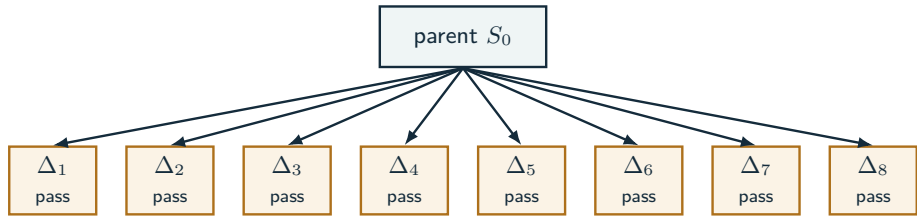
Associate Professor, Holon Institute of Technology (HIT)



* Factory as an atelier:
a workshop for exploration and refinement.

Eight candidates pass. What can we accept?

One prepared state. Eight proposed changes. Eight successful isolated tests.



Selection

Which result is better?

Evidence

How much did we learn?

Composition

Which changes work together?

Each question requires more than the number of successful workers.

Where the pattern matters

Repeated preparation, mutable state, and an outcome we can assess.

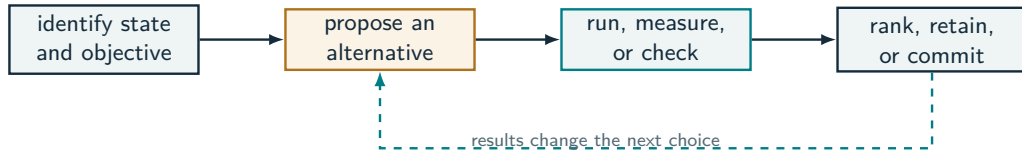
Setting	Candidate	Assessment
Computational biology	Model, parameters, analysis	Fit, robustness, experimental agreement
Scientific computing	Numerical method or hypothesis	Residuals, invariants, independent checks
Model post-training	Policy trajectory	Reward, constraints, held-out performance
Software maintenance	Patch or configuration	Tests, performance, compatibility

Motivating software workloads: Apache Spark, OpenClaw, Apache Airflow, and the Linux kernel.

The runtime pattern transfers; the standard of evidence remains domain-specific.

Development as iterative search

A candidate may be a program, a model, an experiment, or a specification.



- Some outcomes admit proofs or tests; others admit noisy scores or partial rankings.
- A ranking supports a choice only under its stated objective and uncertainty.

The central operation is a controlled alternative followed by an assessment.

The systems hypothesis

Can reusing reached state lower the cost of reaching a fixed quality target?

useful state $\longrightarrow$ new continuations

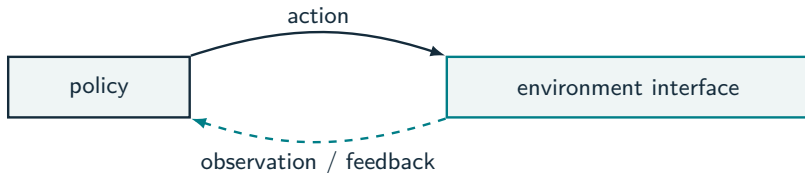
Proposal	A runtime contract for continuation, evidence, and commitment.
Current artifact	A reference evidence reducer and a named-snapshot worker trace.
Open measurement	End-to-end cost advantage at equivalent evaluation quality.

Measure total resources to a declared target, including unsuccessful search.

Eliaz, Evidence-Aware MapReduce for Forkable Compute (2026), preprint.

Environment, sandbox, snapshot, fork

Environment	Defines observations, actions, transitions, feedback, and termination.
Sandbox	Contains an execution and constrains its access and effects.
Snapshot	Records a declared subset of state for a supported restoration procedure.
Fork	Creates a separately evolving continuation from an identified state.



What belongs to the state?

Reproducing a continuation requires more than a filesystem copy.

$$S_t = (F_t, M_t, H_t, R_t ; X_t)$$

F_t : files and artifacts	Repository, dependencies, fixtures, build outputs.
M_t : machine state	Processes, memory, supported devices and runtime state.
H_t : agent history	Messages, tool results, policy identity, pending decisions.
R_t : randomness	Generator state and declared reseeding policy.
X_t : external state	Services, clocks, identities, devices, physical systems.

A capture manifest identifies what is copied, reconstructed, or left external.

A continuation contract

Restoration should preserve the behavior relevant to the task.

$$d(\mathcal{L}(O_{\text{restored}}^\pi), \mathcal{L}(O_{\text{direct}}^\pi)) \leq \varepsilon$$

- Fix the continuation policy π , observation boundary O , and external-state policy.
- Choose a task-relevant distance d and tolerance ε .
- Check declared invariants before measuring search performance.

Example: identical parser fixtures and outputs; bounded timing variation.

A successful restore is the beginning of a fidelity test.

Simulation and the external world

	What can branch?	What still needs validation?
Software simulator	Captured numerical state	Model assumptions and numerical error
Learned world model	Model state and sampled predictions	Prediction error and distribution shift
Physical experiment	Plans and computational analysis	New measurements in the real system



Local rollback cannot rewind a remote service or a wet-lab assay.

One running software experiment

An illustrative parser campaign gives every later abstraction a concrete object.

Prepared parent S_0

- Pinned repository and dependencies
- Loaded corpus and service fixtures
- Identified build and runtime state

Campaign contract

- Eight proposed patch deltas
- Correctness constraints
- Throughput objective and budget

$$S_0 \xrightarrow{\Delta_i} S_i \xrightarrow{\text{evaluate}} (\text{artifact}, \text{receipt})$$

The patch, tests, and parent each need their own identity.

The branch lifecycle



- Source edits must reach the executing program through compilation or reloading.
- Invalidate artifacts affected by the intervention.
- Bind the evaluation receipt to the exact resulting artifact.

Only preparation that remains valid can be reused.

Mechanisms for branching

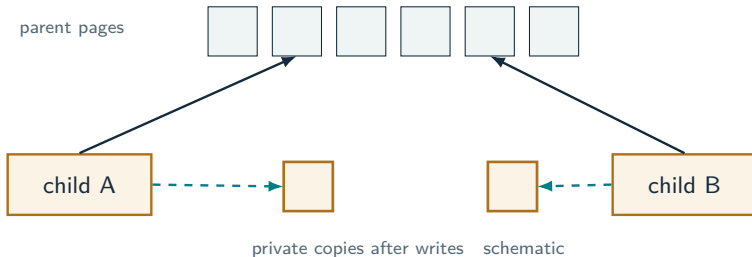
Mechanism	Typical reusable state	Contract to inspect
Process fork	Address space, supported OS resources	Shared kernel, descriptors, runtime constraints
Container restart	Images, volumes, prepared artifacts	Live process state requires extra support
Process checkpoint	Captured processes and resources	Kernel, device, and external dependencies
VM snapshot	Guest memory, CPU and supported devices	Host/device compatibility, identities, effects

Isolation strength and continuation fidelity are separate design choices.

Agache et al., Firecracker, NSDI 2020.

Copy-on-write moves cost to divergence

Children initially share eligible pages; writes create private copies.



- Measure dirty pages, page faults, memory pressure, and useful-work latency.
- Shared caches need explicit ownership, invalidation, and trust scope.

When does branching pay?

$$C_{\text{cold}} = N(P + W + V)$$

$$C_{\text{fork}} = P + H + N(R + D + W + V)$$

P : reusable preparation; H : capture.

R : restore; D : divergence overhead.

W : candidate work; V : evaluation.

$$(N - 1)P > H + N(R + D)$$

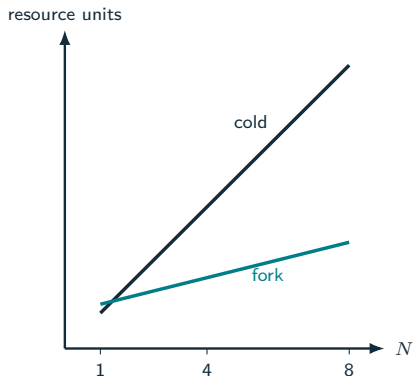
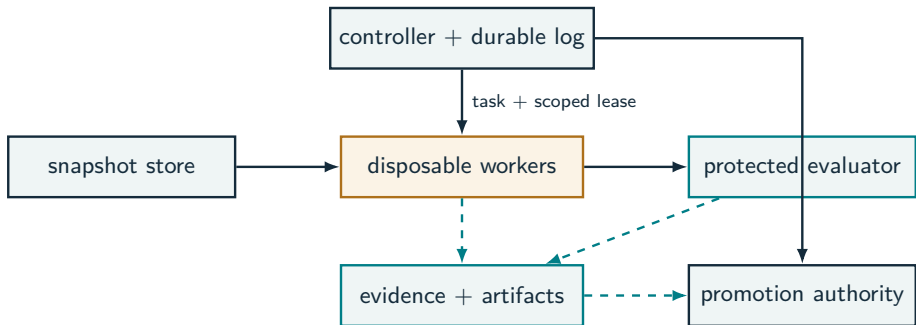


Illustration: $P = 10$, $H = 2$,
 $R + D = 1$, $W + V = 2$.

Analytic illustration, not a measured speedup. Add mechanism-specific costs and contention to the experiment.

A runtime architecture for experiments

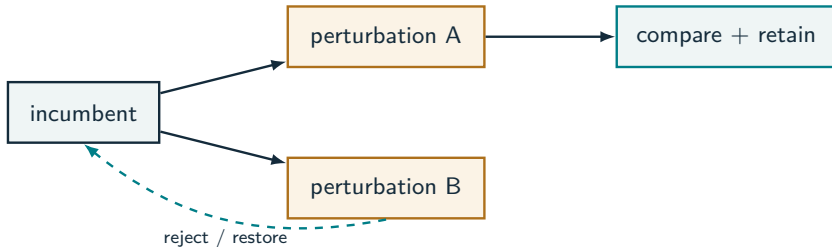


A retry keeps the task identity. A new child receives a fresh scoped capability.

Execution can fail without erasing the experiment or transferring authority.

Stochastic improvement

A preserved parent lets a search policy try alternatives and return.



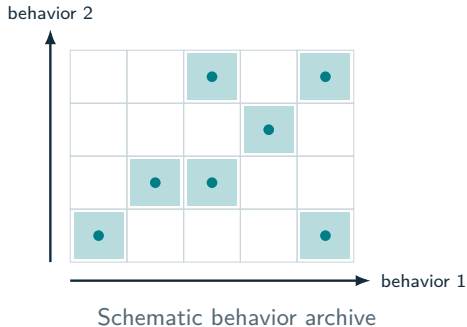
- Propose a change under a fixed objective and constraints.
- Compare with appropriate uncertainty; retain the evidence of failure too.
- Choose acceptance and exploration rules explicitly.

Rollback is a runtime operation; improvement is an algorithmic judgment.

Population search and diversity

Selection need not discard every alternative to the current winner.

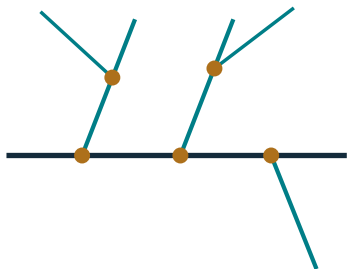
- Mutation explores nearby candidates.
- Crossover proposes a new composition.
- An archive retains different useful behaviors.



Every recombined artifact needs a fresh evaluation.

Mouret & Clune, MAP-Elites (2015).

Branching in biological networks



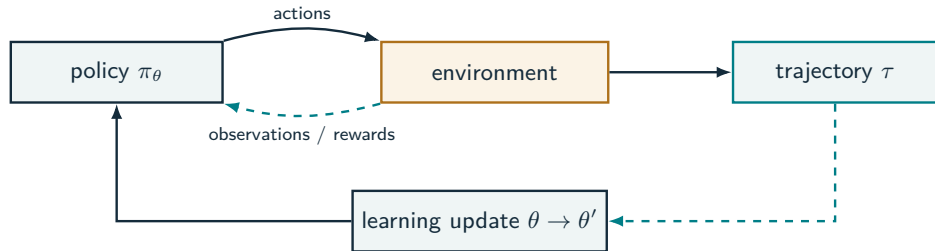
branching-network schematic
not a molecular reconstruction

Local mechanisms shape the structure available to the whole system.

- Actomyosin work connects local connectivity to global morphology.
- Arp2/3 supplies a concrete biological branching mechanism.
- Computational design question: where should branches grow, interact, or stop?

Eliaz et al., Phys. Rev. E 102, 062420 (2020); Liman et al., Arp2/3 and branched actomyosin networks, PNAS (2020).

Reinforcement learning has two loops



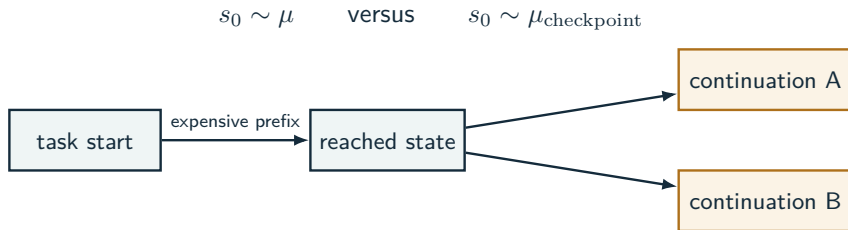
$$J_{\text{RL}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \gamma^t r_t \right]$$

Snapshots can support experience generation; the learner still defines the update.

Sutton & Barto, Reinforcement Learning: An Introduction, 2nd ed. (2018).

Reset changes the learning problem

The distribution of starting states determines which experience is collected.



- Record the origin and selection rule for each checkpoint.
- Evaluate the final policy on the intended task distribution.

Related return-and-explore strategy: [Ecoffet et al., Go-Explore, Nature \(2021\)](#).

Automated research chooses experiments

The next candidate can change the question or the measurement procedure.

Search over artifacts	Which implementation or parameter setting performs better?
------------------------------	--

Search over experiments	Which test distinguishes competing explanations?
--------------------------------	--

Search over tasks	Which challenge exposes a missing capability?
--------------------------	---

hypotheses $\longleftrightarrow$ experiments $\longleftrightarrow$ evidence

A changed research objective starts a new comparison context.

Task-solution coevolution: [Wang et al., POET \(2019\)](#).

Which computation is worth buying?

A controller can spend its remaining budget in several ways.

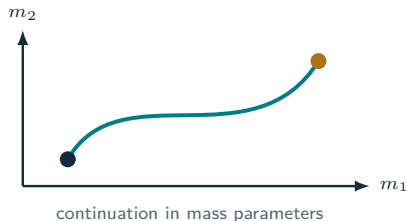
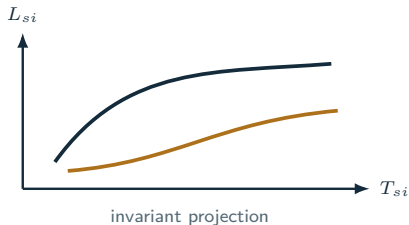
Another candidate	Explore a different region of the search space.
A longer continuation	Resolve a promising partial result.
A discriminating check	Separate explanations that currently look equivalent.
Independent corroboration	Test whether the current evidence shares a failure mode.
A composition test	Determine whether useful changes work together.

Choose by expected decision benefit relative to full cost.

Metareasoning perspective: [Hay et al., Selecting Computations: Theory and Applications \(2012\)](#).

Apparent branches, verified connections

With Ori Chamo: testing connectivity behind an apparent split.



Both diagrams are conceptual schematics, not replotted orbit data.

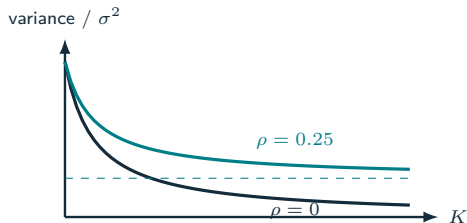
Connected sampled component; distinct stability regimes.

Chamo & Eliaz, Three-body continuation geometry, [ai.viXra:2608.0069](https://arxiv.org/abs/2608.0069) (2026), preprint.

Repeated runs can share an error

Under equal variances and common pairwise correlation:

$$\text{Var}(\bar{Y}_K) = \sigma^2 \left(\rho + \frac{1 - \rho}{K} \right)$$



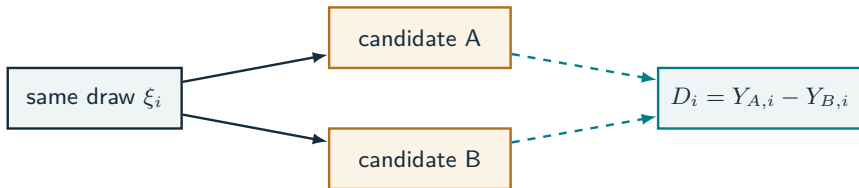
Analytic illustration; $0 \leq \rho \leq 1$.

- Shared data and assumptions can couple outcomes.
- A copied ancestor does not determine the correlation numerically.
- More workers can repeat the same mistaken conclusion.

Shared randomness can improve a comparison

For a difference, positive covariance can reduce uncertainty.

$$\text{Var}(Y_A - Y_B) = \text{Var}(Y_A) + \text{Var}(Y_B) - 2 \text{Cov}(Y_A, Y_B)$$

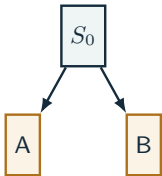


Preserve nuisance variation when comparing; vary failure modes when corroborating.

Glasserman & Yao, Some Guidelines and Guarantees for Common Random Numbers, Management Science (1992).

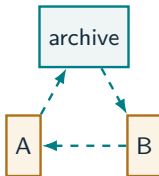
Three graphs describe the computation

Execution ancestry



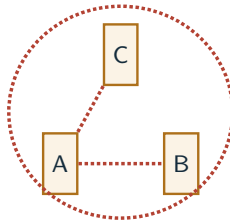
Which state produced this continuation?

Evidence flow



Which observations influenced a decision?

Composition



Which candidate sets need joint validation?

A runtime tree can support a communication network and higher-order interactions.

What does a message contribute?

Information gain is conditional on a question and what is already known.

$$I(H; E_k \mid E_{<k})$$

A small synergy example

Let A, B be independent fair bits and $H = A \oplus B$.

$$I(H; A) = I(H; B) = 0, \quad I(H; A, B) = 1 \text{ bit.}$$

For a branch controller

- Redundant messages can add little.
- Complementary messages can matter jointly.
- Estimation requires a distribution and target.

Message count, Shannon information, and decision utility are different quantities.

Information decomposition perspective: Williams & Beer, Nonnegative Decomposition of Multivariate Information (2010).

An evidence record

A branch returns enough context to interpret its result.

$$r_k = (\hat{\theta}_k, J_k, n_k, \mathcal{E}_k, \mathcal{L}_k, m_k)$$

Estimate and information	$\hat{\theta}_k$, calibrated per-observation J_k , sample count n_k .
Evidence identity	Which observations or data contributed?
Lineage	Which parent, intervention, and continuation produced the result?
Metadata	Objective, policy, evaluator, runtime, and selection context.

A receipt needs provenance; a declaration alone does not authenticate itself.

Eliaz, Evidence-Aware MapReduce for Forkable Compute (2026), preprint.

Evidence-aware reduction

For independent evidence about one common parameter, with calibrated precision:

$$P_k = n_k J_k, \quad P = \sum_k P_k, \quad \hat{\theta} = P^{-1} \sum_k P_k \hat{\theta}_k$$

- Gaussian or local Wald approximation; positive-definite precision.
- Numeric summaries support tree reduction under these assumptions.
- Exact evidence overlap is detectable; hidden dependence may remain.

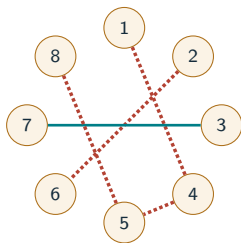
Unresolved dependence blocks a justified claim of added precision.

Eliaz, Evidence-Aware MapReduce for Forkable Compute (2026), preprint.

How eight changes work together

Individual success does not establish the quality of a composition.

$$\Gamma_{ij} = U(\{i, j\}) - U(\{i\}) - U(\{j\}) + U(\emptyset)$$



Illustrative interaction graph

- $U(A)$: fixed objective for change set A from the same parent.
- Eight candidates yield 28 pair tests.
- Higher-order effects can survive pairwise screening.
- Rebuild and evaluate each chosen composition.

Evaluation under adaptive search

A protected evaluator can still teach the search how to overfit its feedback.



- Keep evaluation code and admission records outside worker write access.
- Track queries and selection history.
- Use fresh validation or a justified adaptive evaluation procedure.

Russo & Zou, Controlling Bias in Adaptive Data Analysis Using Information Theory, AISTATS (2016).

The atelier as an institution

A collaboration protocol determines how private trials become shared commitments.

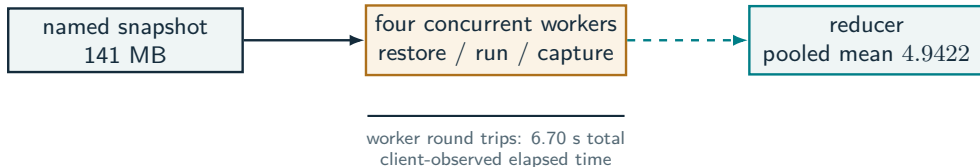
Explore privately	Workers change their own candidates within scoped permissions.
Share evidence	A broker records results, provenance, and decision-relevant messages.
Validate jointly	A new integration trial checks the proposed composition.
Commit explicitly	Current parent, objective, and evaluator identities authorize one acceptance.

A successful old receipt cannot authorize a changed artifact or a stale parent.

Related transaction pattern: [Kung & Robinson, On Optimistic Methods for Concurrency Control, TODS \(1981\).](#)

The implemented execution path

One reported integration trace connects a snapshot to workers and a reducer.



Seed-pinned full-sample mean: 4.9450.

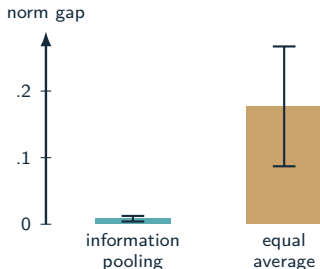
This demonstrates an execution path. Comparative cost benefit remains unmeasured.

Reported in Eliaz, Evidence-Aware MapReduce for Forkable Compute (2026).

What the reducer experiments establish

Unequal logistic shards

Distance to the centralized MLE; lower is closer.



0.0083 ± 0.0042 vs. 0.177 ± 0.090
Mean $\pm$ SD over eight fixed seeds.

Reported synthetic results: [Eliaz \(2026\)](#), [arXiv:2607.09689](#). Error bars show SD, not confidence intervals.

Scope of the checks

- Gaussian summaries match the stated pooling algebra.
- Information weighting helps in this synthetic comparison.
- Forged precision exposes a trust failure; a heuristic is not a robustness theorem.

A maintenance campaign end to end

Proposed Airflow-style campaign: improve task retries without introducing duplicate effects.

Prepare	Pin repository, database fixtures, task graph, and failure scenario.
Explore	Fork candidate implementations; record patch and build identities.
Evaluate	Check retry behavior, compatibility, and timing under injected failures.
Compose	Combine selected changes in a clean integration branch and rerun checks.
Review	Present the artifact, evidence, remaining uncertainty, and rollback plan.

Mock or mediate external effects; a local snapshot cannot undo them.

A test that could reject the hypothesis

Hold tasks, policy, evaluation, and the quality target fixed.

Cold reconstruction	Recreate preparation for each trial.
Cached restart	Reuse immutable artifacts with valid cache identities.
Checkpoint restore	Restore supported state for each continuation.
Forked fan-out	Reuse a reached parent across simultaneous continuations.

Measure resource use and time to target, peak memory, failures, fidelity, and unintended effects.

Report when the target is not reached within budget.

Ablations that explain the outcome

Vary	Observe	Question
Prefix cost and dirty pages	Resource use and memory	When does reuse pay?
No sharing / archive / routing	Quality at equal full cost	When does communication help?
Evidence overlap and coupling	Calibration and error	Does the reducer overcount?
Evaluation budget	False acceptance and time to target	Where should compute go?
Composition order and size	Joint success and regressions	Which interactions matter?

Separate a runtime gain from a search-policy or evaluation advantage.

Research questions at the runtime boundary

1. **Where should a checkpoint live?**

Choose useful states jointly with exploration cost and restore fidelity.

2. **Which evidence should travel?**

Route complementary observations while preserving selection and dependence history.

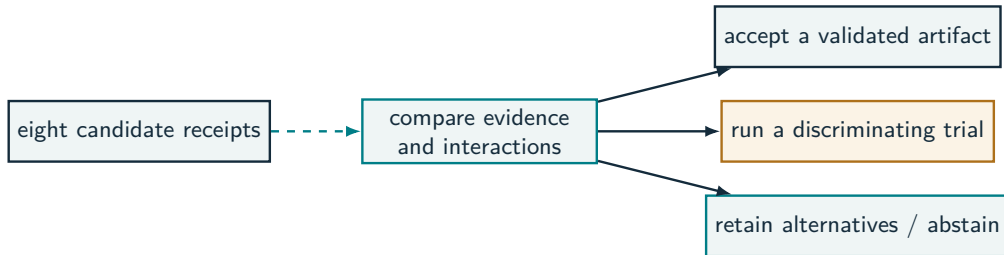
3. **How should tasks and solutions evolve together?**

Generate new challenges without losing the meaning of progress or validation.

The runtime interface becomes part of the learning and experimental design.

Revisiting the eight candidates

The outcome of a campaign is a justified decision with a recorded scope.



- Select with an objective and uncertainty.
- Combine through a fresh integration experiment.
- Preserve what the evidence does not yet settle.

What should a runtime expose to a learning system?

State that can be continued.

Interventions whose effects can be assessed.

Costs that include the whole experiment.

Evidence with provenance and uncertainty.

Authority to make an explicit commitment.

Which computation would most improve the next decision?